



机器学习&深度学习 基础

1. 数学基础
2. 机器学习
3. 多层感知机

1. 数学基础 导数



- 导数：函数在点 x 处的导数定义如下，导数在机器学习和深度学习中的应用是求解目标函数的极值

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

- 基本函数的求导公式

$$(x^a)' = ax^{a-1}$$

$$(e^x)' = e^x$$

$$(a^x)' = a^x \ln a$$

$$(\ln x)' = \frac{1}{x}$$

- 四则运算的求导公式

$$(f(x) + g(x))' = f'(x) + g'(x)$$

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x)$$

$$\left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)g(x) - f(x)g'(x)}{g^2(x)}$$

- 复合函数的求导公式

$$(f(g(x)))' = f'(g)g'(x)$$



➤ 1. 数学基础 导数

- 高阶导数：对导数继续求解，可以得到高阶导数，二阶导数是一阶导数的导数，记为

$$f''(x)$$

n阶导数记为

$$f^{(n)}(x)$$

- 一元函数的泰勒展开

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \dots + \frac{1}{n!}f^{(n)}(x_0)(x - x_0)^n \dots$$

- 导数与函数的性质

- 一阶导数决定函数的单调性，导数大于0，函数单调增，导数小于0，函数单调减
- 极值定理 一个可导函数在极值点处导数必定为0。导数为0的点称为函数的驻点
- 二阶导数决定函数的凹凸性：二阶导数大于0，函数为凸函数；二阶导数小于0，函数为凹函数。二阶导数为0的点为函数的拐点

- 一元函数的极值判别法则

- 如果在x点处导数为0，即为函数的驻点，则
- 如果二阶导数大于0，x为函数的极小值点
- 如果二阶导数小于0，x为函数的极大值点
- 如果二阶导数为0，则情况不定



➤ 1. 数学基础 导数

- 梯度：多元函数对各个自变量的一阶偏导数构成的向量

- 梯度算子 (Nabla) : $\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^T$

- 偏导数：导数对多元函数的推广

$$\frac{\partial f}{\partial x_i} = \lim_{\Delta x_i \rightarrow 0} \frac{f(x_1, \dots, x_i + \Delta x_i, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{\Delta x_i}$$

- 高阶偏导数：类似于高阶导数，可以定义高阶偏导数，但情况更为复杂

- 二阶偏导数

$$\frac{\partial^2 f}{\partial x \partial y} \quad \frac{\partial^2 f}{\partial y \partial x} \quad \frac{\partial^2 f}{\partial x^2} \quad \frac{\partial^2 f}{\partial y^2}$$

- 一般情况下，混合二阶偏导数与求导次序无关，即

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$$

- 多元函数的极值判别法则
 - 如果Hessian矩阵正定，函数在该点有极小值
 - 如果Hessian矩阵负定，函数在该点有极大值
 - 如果Hessian矩阵不定，则为鞍点，不是极值点
- 矩阵正定的定义： $x^T A x > 0$
- 矩阵正定的判别法则：矩阵的特征值全大于0，矩阵的所有顺序主子式都大于0，矩阵合同于单位阵
- 矩阵与向量求导： $\nabla w^T x = w$, $\nabla x^T A x = (A + A^T)x$, $\nabla^2 x^T A x = A + A^T$
- 多元函数的泰勒展开： $f(x) = f(x_0) + (\nabla f(x_0))^T (x - x_0) + \frac{1}{2} (x - x_0)^T H (x - x_0) + o(\|x - x_0\|^2)$

1. 数学基础 矩阵



- 雅克比矩阵: 对于如下向量到向量的映射函数:

$$y_i = f_i(x)$$

雅克比矩阵为所有因变量对所有自变量的偏导数构成的矩阵

$$\begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial y_m}{\partial x_1} & \frac{\partial y_m}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

- 雅克比矩阵的每一行为一个多元函数的梯度

$$\begin{aligned} u &= x^2 + 2xy + z \\ v &= x - y^2 + z^2 \end{aligned}$$

- 张量: 相当于编程语言中的多维数组, n阶张量矩阵是2阶张量, 向量是1阶张量; 例如RGB彩色图像就是3阶张量。深度学习中并不需要张量的复杂知识, 只有基本的运算

- Hessian矩阵 为多元函数的二阶偏导数构成的矩阵

$$\begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- 是一个对称矩阵, 相当于一元函数的二阶导数

$$f(x, y, z) = 2x^2 - xy + y^2 - 3z^2$$

- 乘法结合律: $(AB)C = A(BC)$
- 分配率: $(A + B)C = AC + BC$ 、 $A(B + C) = AB + AC$
- 乘法不满足交换律 $AB \neq BA$
- 逆矩阵 $(AB)^{-1} = B^{-1}A^{-1}$ 、 $(A^{-1})^{-1} = A$ 、 $(A^T)^{-1} = (A^{-1})^T$

➤ 随机事件的概率

- 条件概率：一随机事件发生的前提下另一随机事件发生的概率

$$p(b|a) = \frac{p(a,b)}{p(a)} \quad p(a|b) = \frac{p(a,b)}{p(b)}$$

- 贝叶斯公式

$$p(a|b) = \frac{p(a)p(b|a)}{p(b)}$$

➤ 随机事件的独立性

- 两个随机事件独立

$$p(a,b) = p(a)p(b)$$

- 多个随机事件独立

$$p(a_1, \dots, a_n) = \prod_{i=1}^n p(a_i)$$



- 随机变量：取每个值关联一个概率值的变量
- 离散型随机变量：取值只有有限种情况，或者无限可列种情况，描述离散型随机变量的是概率分布

$$\begin{aligned} p(x = x_i) &\geq 0 \\ \sum p(x = x_i) &= 1 \end{aligned}$$

- 连续型随机变量，取值为无限不可列种情况，即一个区间内的实数描述连续型随机变量的是概率密度函数和分布函数。
- ✓ 概率密度函数必须满足

$$\begin{aligned} f(x) &\geq 0 \\ \int_{-\infty}^{+\infty} f(x) dx &= 1 \end{aligned}$$

- ✓ 分布函数定义为 $F(y) = p(x \leq y) = \int_{-\infty}^y f(x) dx$

- 数学期望: 概率意义上的均值

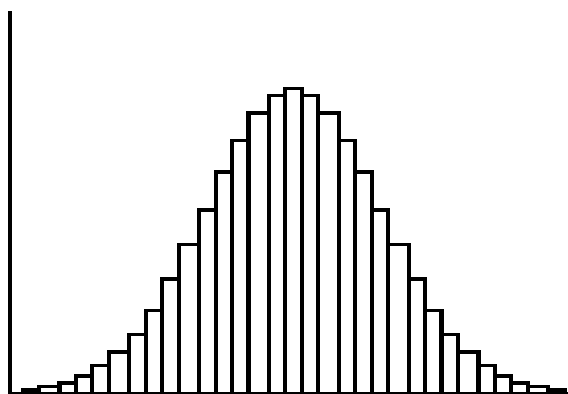
- 对于离散型随机变量，定义为 $E(x) = \sum x_i p(x_i)$

- 对于连续型随机变量，定义为 $E(x) = \int_{-\infty}^{+\infty} x f(x) dx$

- 方差: 反映数据偏离数学期望的程度，即波动程度

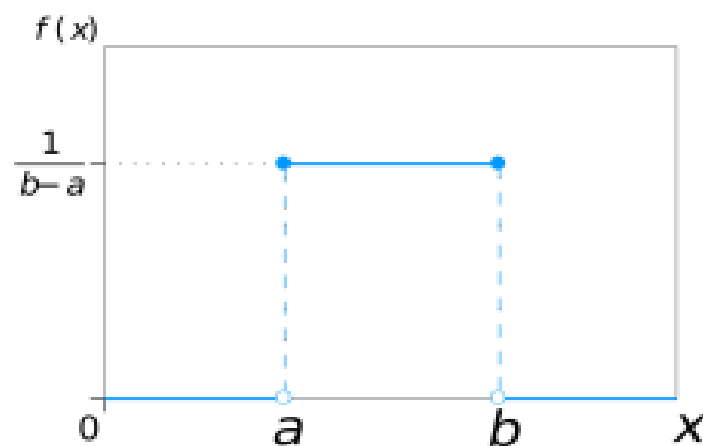
- 对于离散型随机变量，定义为 $D(x) = \sum (x_i - E(x))^2 p(x_i)$

- 对于连续型随机变量，定义为 $D(x) = \int_{-\infty}^{+\infty} (x - E(x))^2 f(x) dx$



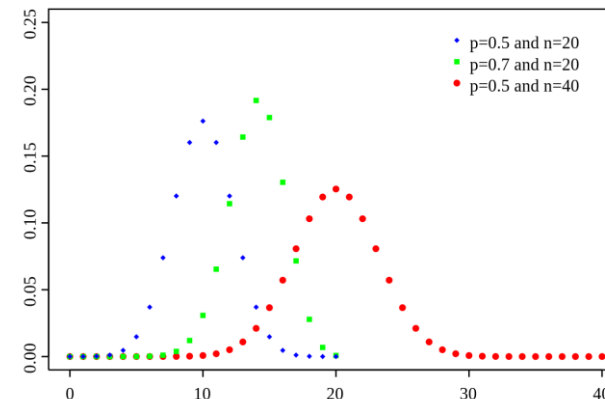
正态分布

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



均匀分布

$$f(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & x < a, x > b \end{cases}$$



二项分布

$$\begin{aligned} p(x=1) &= p \\ p(x=0) &= 1-p \end{aligned}$$



➤ 随机变量：向量的每一个分量都是随机变量

- 离散型随机向量，用分布律描述 $p(x = x_i)$
- 连续型随机向量，用联合概率密度函数描述

$$f(x_1, x_2) \geq 0$$
$$\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f(x_1, x_2) dx_1 dx_2 = 1$$

➤ 随机变量的独立性：

- 两个随机变量相互独立

$$f(x_1, x_2) = f(x_1)f(x_2)$$

- n 个随机变量相互独立

$$f(x_1, \dots, x_n) = \prod_{i=1}^n f(x_i)$$

➤ 边缘概率

- 离散型随机向量 $p(x = x_i) = \sum_y p(x = x_i, y = y_j)$

• 连续型随机向量-边缘密度函数

$$f(x_1) = \int_{-\infty}^{+\infty} f(x_1, x_2) dx_2$$
$$f(x_2) = \int_{-\infty}^{+\infty} f(x_1, x_2) dx_1$$

➤ 协方差，描述两个随机变量直接的线性相关性，定义为

$$\text{cov}(x_1, x_2) = E((x_1 - E(x_1))(x_2 - E(x_2)))$$

计算公式可以简化为

$$\text{cov}(x_1, x_2) = E(x_1 x_2) - E(x_1)E(x_2)$$



➤ 常见的多维概率分布

- 均匀分布

- 正态分布

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$$

➤ 最大似然估计

- 根据一组样本确定概率密度函数的参数

$$p(\mathbf{x}; \theta) \quad \mathbf{x}_i, i = 1, \dots, l$$

- 这组样本是独立同分布的，它们共同出现的概率为

$$L(\theta) = \prod_{i=1}^l p(\mathbf{x}_i; \theta)$$

- 既然这组样本出现了，就应该让它们的概率最大化

$$\max \prod_{i=1}^l p(\mathbf{x}_i; \theta)$$

- 乘积的求导比较复杂，可以取对数，二者等价

$$\ln L(\theta) = \ln \prod_{i=1}^l p(\mathbf{x}_i; \theta) = \sum_{i=1}^l \ln p(\mathbf{x}_i; \theta)$$

- 最后归结为求解如下极大值问题

$$\max \sum_{i=1}^l \ln p(\mathbf{x}_i; \theta)$$

1. 数学基础 最优化中的基本概念



➤ 最优化中的基本概念：

- 目标函数
- 优化变量
- 可行域
- 等式约束
- 不等式约束
- 局部极小值
- 全局极小值

- 工程上一般用迭代法实现，迭代法的基本思路为构造一个序列，收敛到极值点处
- 即保证下面的极限成立

$$\lim_{k \rightarrow +\infty} \nabla f(\mathbf{x}_k) = 0$$

- 问题的关键是如何根据当前点找到下一个点

$$\mathbf{x}_{k+1} = h(\mathbf{x}_k)$$

$$f(x, y) = x^3 - 2x^2 + e^{xy} - y^3 + 10y^2 + 100 \sin(xy)$$

$$\begin{cases} 3x^2 - 4x + ye^{xy} + 100y \cos(xy) = 0 \\ xe^{xy} - 3y^2 + 20y + x \cos(xy) = 0 \end{cases}$$

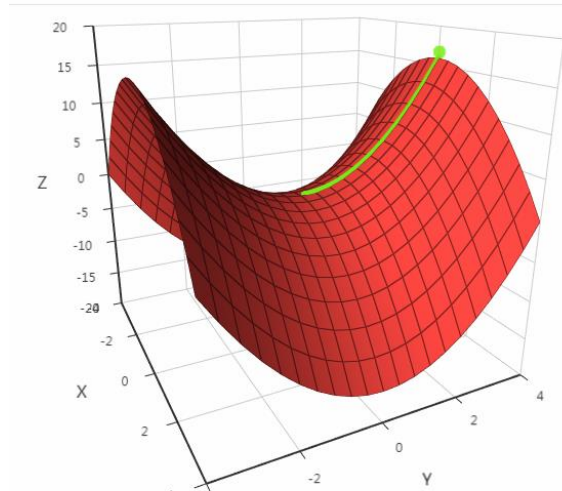
- 梯度下降法：沿着梯度的反方向进行迭代，只要没有到达驻点处，每次迭代时函数值下降

根据泰勒展开得到 $x_{k+1} = x_k - \gamma \nabla f(x_k)$

- 梯度下降法的实现细节问题：初始值的设定，步长的选择，迭代终止的判定规则，在深度学习中被广为使用，有很多变种

- 数值优化算法面临的问题

- 局部极小值问题
- 鞍点问题





1. 数学基础 判别模型

直接得到分类函数，或者直接得到类后验概率

$$y = f(x) \quad p(y|x)$$

生成模型：先对特征向量与标签的联合概率建模，然后计算类后验概率

$$p(x, y) = p(x|y)p(y)$$

另外一种定义：判别模型是对类后验概率建模-研究数据怎么进行分类

$$p(y|x)$$

生成模型是对类条件概率建模-研究数据是怎么生成的

$$p(x|y)$$

➤ 什么是机器学习

- 对于有些问题，我们无法用确定的逻辑编程实现。如图像识别，语音识别，垃圾邮件分类
- 机器学习的核心思想是模拟人的学习能力，从样本数据中学习，得到经验/模型，再进行预测。这是一种数据驱动的方法

➤ 机器学习中的重要概念

- 样本-机器学习算法处理的数据
- 特征向量- 人工构造的用于描述一个样本的向量，如颜色、形状等
- 预测函数- 实现从样本的特征向量到预测值的映射
- 目标函数- 训练时使用，目的是确定模型的参数



2. 机器学习 机器学习算法的分类

- 监督信号：人工为样本打上的**标签 (label)**，如图像的类别，是26个英文字母中的哪一个
 - 按照样本是否有标签值，将机器学习算法分为有监督学习与无监督学习
 - 有监督学习事先用带有标签的样本进行训练，然后用得到的模型预测。
 - 无监督学习则直接对数据进行预测，样本不带有人工标注的标签值

➤ 有监督学习 根据样本学习得到一个映射函数 $y = f(x)$

- 分类问题 确定样本的类别，如人脸识别，字符识别，语音识别。类别标签是整数编号 $\mathbb{R}^n \rightarrow \mathbb{Z}$
- 回归问题 确定一个实数值，如根据一个人的学历、工作年限、行业等信息预测他/她的收入 $\mathbb{R}^n \rightarrow \mathbb{R}$

➤ 无监督学习

聚类问题 - 将一批样本分成多给类，即多个不相交的子集，每个样本属于其中的一个子集。类别没有事先定义，而是由算法确定

数据降维问题 - 将一个向量变换到低维空间中

➤ 强化学习 根据当前的状态确定要执行的动作，以达到某一目标，抽象来说，是最大化累计回报



2. 机器学习 监督学习

➤ 监督学习 (Supervised Learning) 表示机器学习的数据是带标记的, 这些标记可以包括数据类别、数据属性及特征点位置等。这些标记作为预期效果, 不断修正机器的预测结果。

具体实现过程是: 通过大量带有标记的数据来训练机器, 机器将预测结果与期望结果进行比对; 之后根据比对结果来修改模型中的参数, 再一次输出预测结果; 然后将预测结果与期望结果进行比对, 重复多次直至收敛, 最终生成具有一定鲁棒性的模型来达到智能决策的能力。

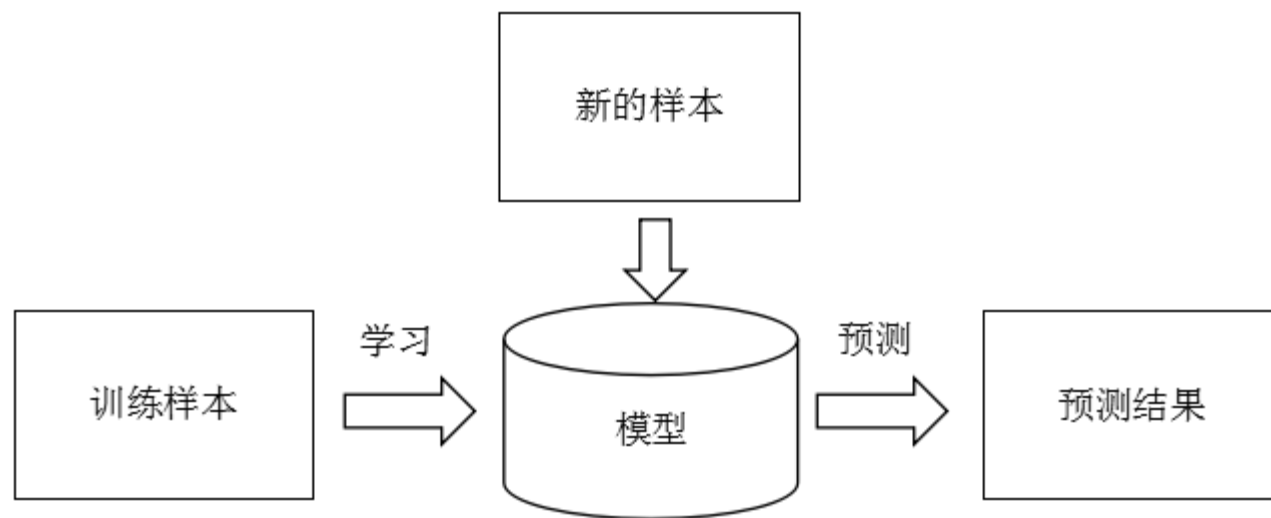
常见的监督学习有分类和回归:

- 分类 (Classification) 是将一些实例数据分到合适的类别中, 它的预测结果是离散的。
 - 分类的例子包括垃圾邮件检测、客户流失预测、情感分析、犬种检测等。
 - 回归 (Regression) 是将数据归到一条“线”上, 即为离散数据生产拟合曲线, 因此其预测结果是连续的。
 - 回归的例子包括房价预测、股价预测、身高-体重预测等。
- 算法举例: 常见的有监督机器学习算法包括支持向量机(Support Vector Machine, SVM), 朴素贝叶斯(Naive Bayes), 逻辑回归(Logistic Regression), K近邻(K-Nearest Neighborhood, KNN), 决策树(Decision Tree), 随机森林(Random Forest), AdaBoost以及线性判别分析(Linear Discriminant Analysis, LDA)等。深度学习(Deep Learning)也是大多数以监督学习的方式呈现

2. 机器学习 有监督学习的一般流程



厦门大学
XIAMEN UNIVERSITY



2. 机器学习 无监督学习



- 无监督学习 (Unsupervised Learning) 表示机器学习的数据是没有标记的。机器从无标记的数据中探索并推断出潜在的联系。常见的无监督学习有聚类和降维
 - 聚类 (Clustering) 由于事先不知道数据类别, 因此只能通过分析数据样本在特征空间中的分布, 例如基于密度或基于统计学概率模型等, 从而将不同数据分开, 把相似数据聚为一类。
 - 降维 (Dimensionality Reduction) 是将数据的维度降低。

例如描述一个西瓜, 若只考虑外皮颜色、根蒂、敲声、纹理、大小及含糖率这6个属性, 则这6个属性代表了西瓜数据的维度为6。进一步考虑降维的工作, 由于数据本身具有庞大的数量和各种属性特征, 若对全部数据信息进行分析, 将会增加训练的负担和存储空间。因此可以通过主成分分析等其他方法, 考虑主要影响因素, 舍弃次要因素, 从而平衡准确度与效率。

- 强化学习 (Reinforcement Learning) 强化学习是带有激励机制的：
- 如果机器行动正确，将施予一定的“正激励”；如果行动错误，同样会给出一个惩罚（也可称为“负激励”）。在这种情况下，机器将会考虑如何在一个环境中行动才能达到激励的最大化，具有一定的动态规划思想。
- 在贪吃蛇游戏中，贪吃蛇需要通过不断吃到“食物”来加分。为了不断提高分数，贪吃蛇需要考虑在自身位置上如何转向才能吃到“食物”，这种学习过程便可理解为一种强化学习。
- 强化学习最为火热的一个应用就是谷歌 AlphaGo 的升级品——AlphaGo Zero。AlphaGo Zero 舍弃了先验知识，不再需要人为设计特征，直接将棋盘上黑、白棋子的摆放情况作为原始数据输入到模型中，机器使用强化学习来自我博弈，不断提升自己从而最终出色完成下棋任务。在没有人类的经验和指导下，深度强化学习依然能够出色地完成指定任务。

2. 机器学习



- 要想具有更强的智慧，除了拥有大量的数据以外还要有好的经验总结方法。深度学习就是一种实现这种机器学习的优秀技术。深度学习本身是神经网络算法的衍生。作为深度学习父类的机器学习，是人工智能的核心，它属于人工智能的一个分支。
- 深度学习是新兴的机器学习研究领域，旨在研究如何从数据中自动地提取多层特征表示，其核心思想是通过数据驱动的方式，采用一系列的非线性变换，从原始数据中提取由低层到高层、由具体到抽象、由一般到特定语义的特征。
- 深度学习不仅改变着传统的机器学习方法，也影响着人类感知的理解，迄今已在语音识别、图像理解、自然语言处理和视频推荐等应用领域引发了突破性的变革。



2. 机器学习 机器学习常见算法



厦门大学
XIAMEN UNIVERSITY

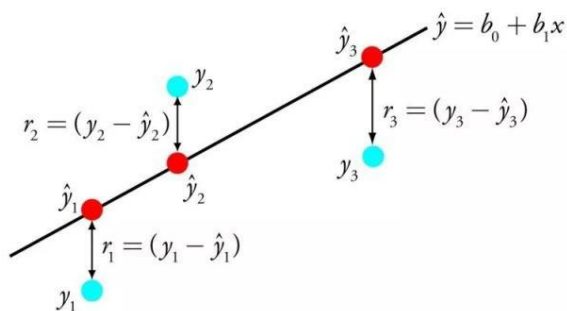
Machine Learning Algorithms (sample)

	Unsupervised 做辅助	Supervised 主要
Continuous	- Clustering & Dimensionality Reduction - SVD 降维 - PCA 降维 - K-means 大部分聚类用	- Regression - Linear - Polynomial - Decision Trees - Random Forests
Categorical	- Association Analysis 关联规则 - Apriori - FP-Growth - Hidden Markov Model 很少用，循环神经网络替换	- Classification - KNN 较少用 - Trees 常用 - Logistic Regression 常用 - Naive-Bayes NLP - SVM 中小型数据集表现好

2. 机器学习 机器学习算法 1. 线性回归



- 在统计学和机器学习领域，线性回归可能是最广为人知也最易理解的算法之一。
- 预测建模主要关注的是在牺牲可解释性的情况下，尽可能最小化模型误差或做出最准确的预测。
- 线性回归模型被表示为一个方程式，它为输入变量找到特定的权重（即系数 B ），进而描述一条最佳拟合了输入变量 (x) 和输出变量 (y) 之间关系的直线。



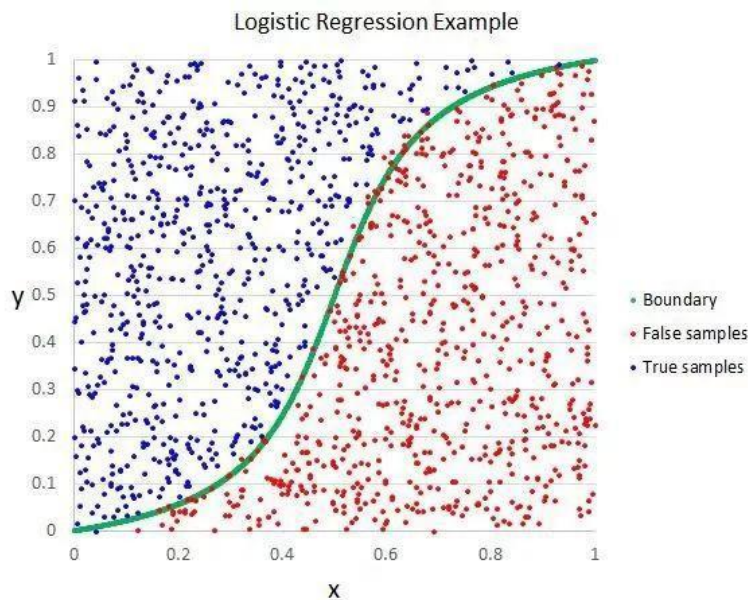
$$y = B0 + B1 * x$$

- 在给定输入值 x 的条件下预测 y ，线性回归学习算法的目的是找到系数 $B0$ 和 $B1$ 的值。
- 可以使用不同的技术来从数据中学习线性回归模型，例如普通最小二乘法的线性代数解和梯度下降优化。
- 经验规则：我们可以删除非常类似（相关）的变量，并尽可能移除数据中的噪声。
- 线性回归是一种运算速度很快的简单技术，也是一种适合初学者尝试的经典算法。

2. 机器学习 机器学习算法2. Logistic 回归

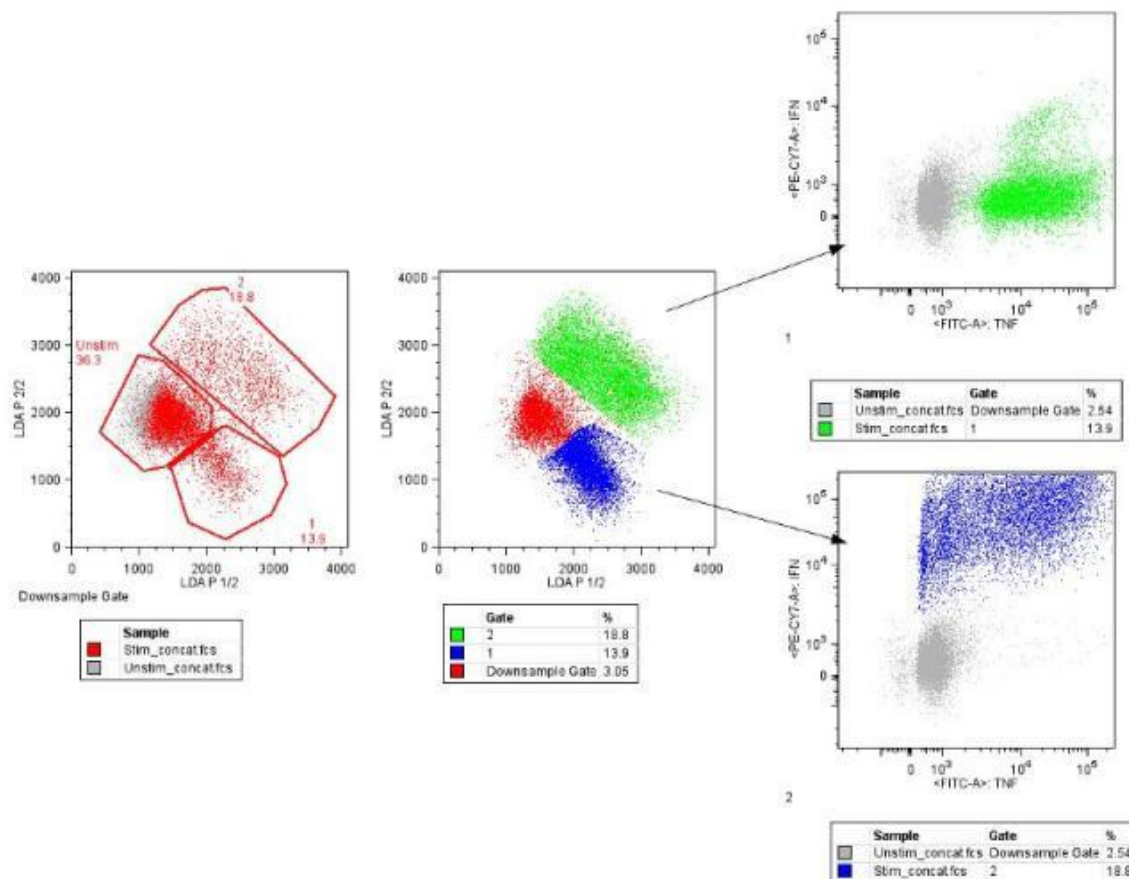


- Logistic 回归是机器学习从统计学领域借鉴过来的另一种技术。它是二分类问题的首选方法。
- 像线性回归一样，Logistic 回归的目的也是找到每个输入变量的权重系数值。但不同的是，Logistic 回归的输出预测结果是通过一个叫作「logistic 函数」的非线性函数变换而来的。
- logistic 函数的形状看起来像一个大的「S」，它会把任何值转换至 0-1 的区间内。因为我们可以把一个规则应用于 logistic 函数的输出，从而得到 0-1 区间内的捕捉值（例如，将阈值设置为 0.5，则如果函数值小于 0.5，则输出值为 1），并预测类别的值。



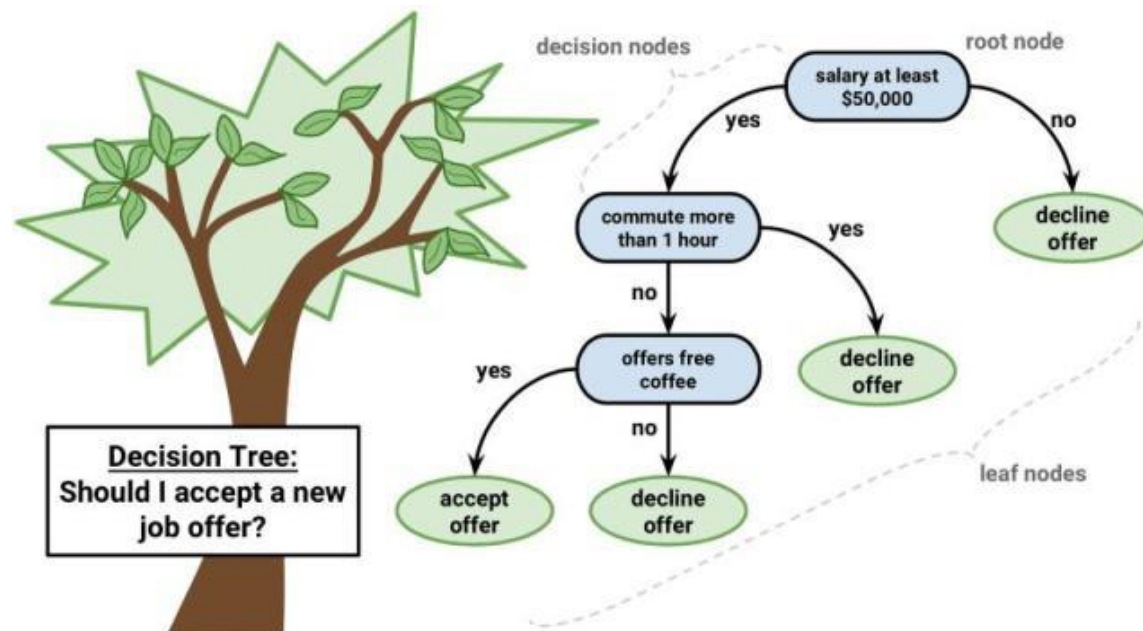
- Logistic 回归的预测结果也可以用作给定数据实例属于类 0 或类 1 的概率。这对于需要为预测结果提供更多理论依据的问题非常有用。
- 与线性回归类似，当删除与输出变量无关以及彼此之间非常相似（相关）的属性后，Logistic 回归的效果更好。该模型学习速度快，对二分类问题十分有效

2. 机器学习 3. 线性判别分析



- Logistic 回归是一种传统的分类算法，它的使用场景仅限于二分类问题。
- 如果有两个以上的类，那么线性判别分析算法（LDA）是首选的线性分类技术。LDA 的表示方法非常直接。它包含为每个类计算的数据统计属性。
 - 每个类的均值
 - 所有类的方差
- 预测结果是通过计算每个类的判别值、并将类别预测为判别值最大的类而得出的。该技术假设数据符合高斯分布（钟形曲线），因此最好预先从数据中删除异常值

2. 机器学习 机器学习算法 4. 分类和回归树

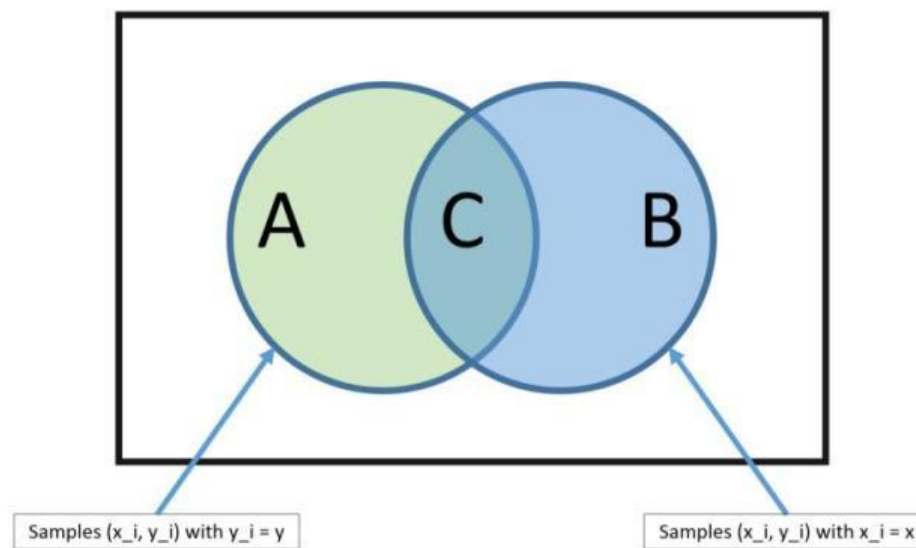


- 决策树是一类重要的机器学习预测建模算法
- 决策树可以被表示为一棵二叉树。
- 这种二叉树与算法设计和数据结构中的二叉树是一样的。
- 每个节点都代表一个输入变量 (x) 和一个基于该变量的分叉点 (假设该变量是数值型的) 决策树的叶子结点包含一个用于做出预测的输出变量 (y) 。
- 预测结果是通过在树的各个分叉路径上游走，直到到达一个叶子结点并输出该叶子结点的类别值而得出。
- 决策树的学习速度很快，做出预测的速度也很快。它们在大量问题中往往都很准确，而且不需要为数据做任何特殊的预处理准备

2. 机器学习 5. 朴素贝叶斯



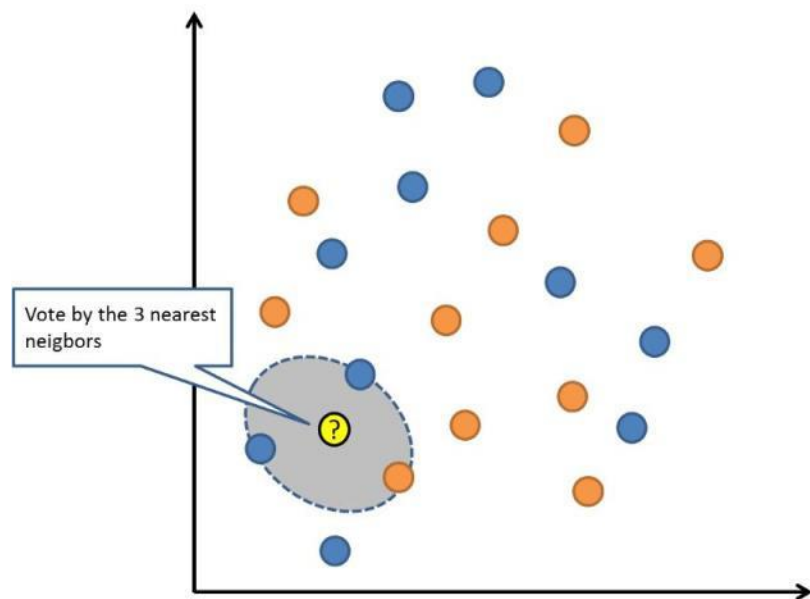
- 朴素贝叶斯是一种简单而强大的预测建模算法
- 该模型由两类可直接从训练数据中计算出来的概率组成：
 - 1) 数据属于每一类的概率；
 - 2) 给定每个 x 值，数据从属于每个类的条件概率。
- 一旦这两个概率被计算出来，就可以使用贝叶斯定理，用概率模型对新数据进行预测。当你的数据是实值的时候，通常假设数据符合高斯分布（钟形曲线），这样你就可以很容易地估计这些概率
- 朴素贝叶斯之所以被称为「朴素」，是因为它假设每个输入变量相互之间是独立的。这是一种很强的、对于真实数据并不现实的假设。不过，该算法在大量的复杂问题中十分有效。



2. 机器学习 算法6- K 最近邻算法



➤ K 最近邻 (KNN) 算法是非常简单而有效的。KNN 的模型表示就是整个训练数据集

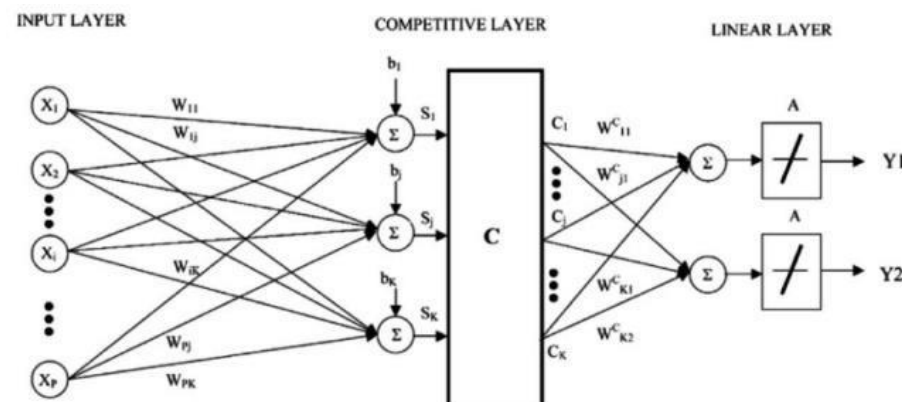


- 对新数据点的预测结果是通过在整个训练集上搜索与该数据点最相似的 K 个实例 (近邻) 并且总结这 K 个实例的输出变量而得出的。对于回归问题来说, 预测结果可能就是输出变量的均值; 而对于分类问题来说, 预测结果可能是众数 (或最常见的) 的类的值。
- 关键之处在于如何判定数据实例之间的相似程度。如果你的数据特征尺度相同 (例如, 都以英寸为单位), 那么最简单的度量技术就是使用欧几里得距离, 你可以根据输入变量之间的差异直接计算出该值
- KNN 可能需要大量的内存或空间来存储所有数据, 但只有在需要预测时才实时执行计算 (或学习)。随着时间的推移, 你还可以更新并管理训练实例, 以保证预测的准确率

➤ 使用距离或接近程度的度量方法可能会在维度非常高的情况下 (有许多输入变量) 崩溃, 这可能会对算法在你的问题上的性能产生负面影响。这就是所谓的维数灾难, 我们应该仅仅使用那些与预测输出变量最相关的输入变量

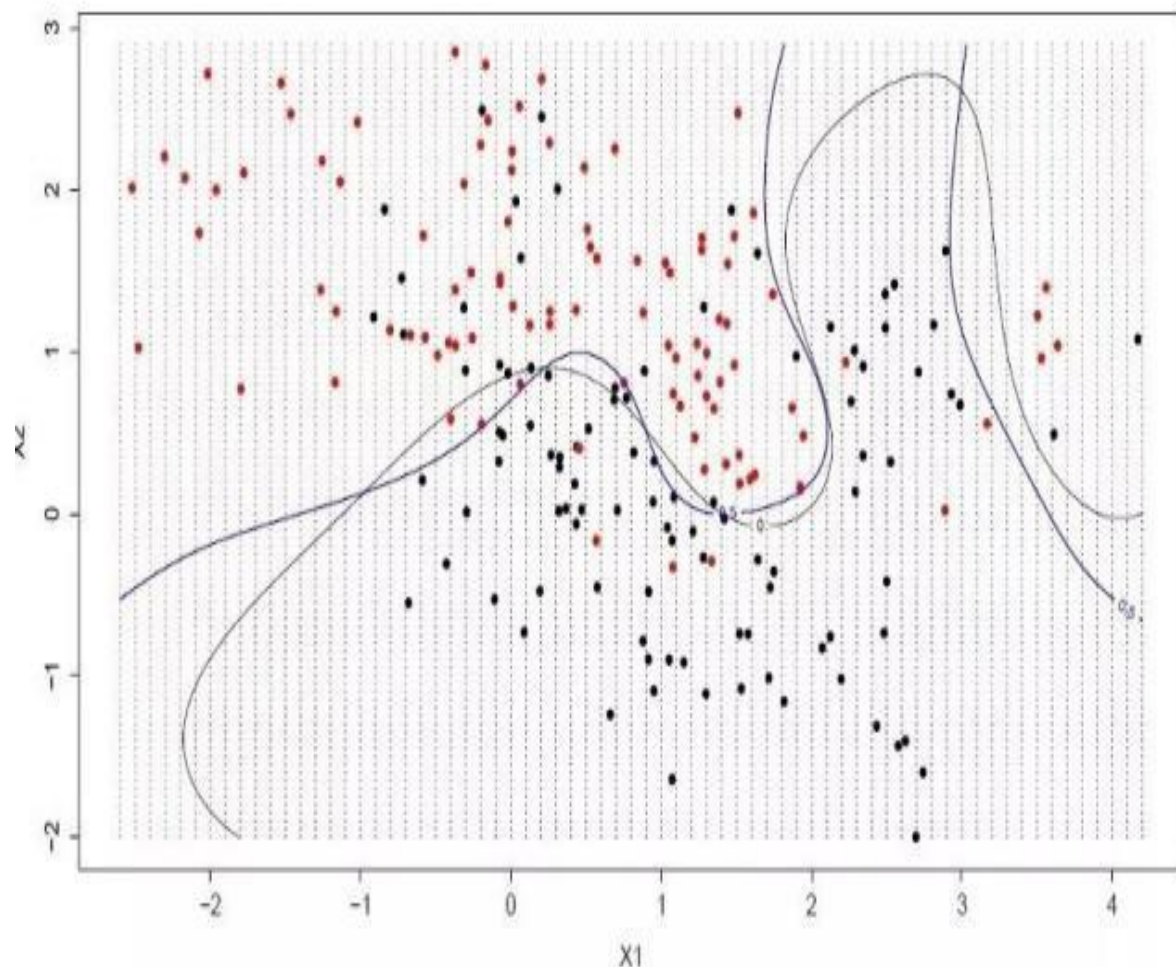
2. 机器学习 算法-7. 学习向量量化

- KNN 算法的一个缺点是，你需要处理整个训练数据集。而学习向量量化算法（LVQ）允许选择所需训练实例数量，并确切地学习这些实例。
- LVQ 的表示是一组码本向量。它们在开始时是随机选择的，经过多轮学习算法的迭代后，最终对训练数据集进行最好的总结。通过学习，码本向量可被用来像 K 最近邻那样执行预测。通过计算每个码本向量与新数据实例之间的距离，可以找到最相似的邻居（最匹配的码本向量）。然后返回最匹配单元的类别值（分类）或实值（回归）作为预测结果。如果将数据重新放缩放到相同的范围中（例如 0 到 1 之间），就可以获得最佳的预测结果。
- 如果KNN 能够在数据集上得到不错的预测结果，那么不妨试一试 LVQ 技术，它可以减少对内存空间的需求，不需要像 KNN 那样存储整个训练数据集。





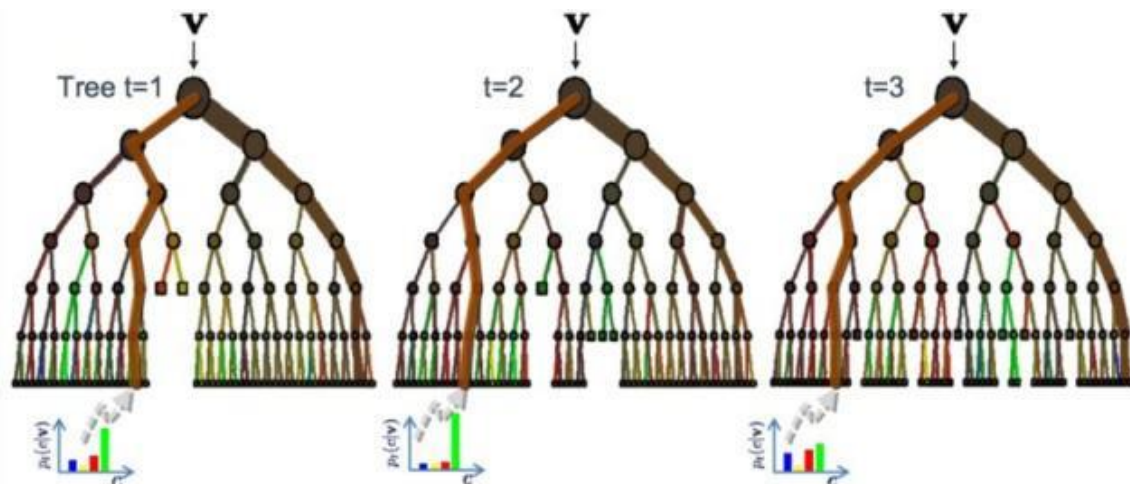
2. 机器学习 算法-8. 支持向量机



- 超平面是一条对输入变量空间进行划分的「直线」。
- 支持向量机会选出一个将输入变量空间中的点按类（类 0 或类 1）进行最佳分割的超平面。
- 在二维空间中，你可以把他想象成一条直线，假设所有输入点都可以被这条直线完全地划分开来。
- SVM 学习算法旨在寻找最终通过超平面得到最佳类别分割的系数
- 超平面与最近数据点之间的距离叫作间隔（margin）。能够将两个类分开的最佳超平面是具有最大间隔的直线。
- 只有这些点与超平面的定义和分类器的构建有关，这些点叫作支持向量，它们支持或定义超平面。

支持向量机可能是目前可以直接使用的最强大的分类器之一

2. 机器学习 算法-9. 袋装法和随机森林



The ensemble model

Forest output probability
$$p(c|v) = \frac{1}{T} \sum_t p_t(c|v)$$

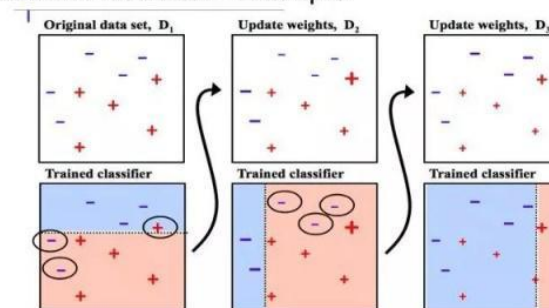


- 自助法是一种从数据样本中估计某个量（例如平均值）的强大统计学方法。
- 需要在数据中取出大量的样本，计算均值，然后对每次取样计算出的均值再取平均，从而得到对所有数据的真实均值更好的估计
- Bagging 使用了相同的方法。但是最常见的做法是使用决策树，而不是对整个统计模型进行估计。
- Bagging 会在训练数据中取多个样本，然后为每个数据样本构建模型。
- 当你需要对新数据进行预测时，每个模型都会产生一个预测结果，Bagging 会对所有模型的预测结果取平均，以便更好地估计真实的输出值。

2. 机器学习 -10. Boosting 和 AdaBoost

- Boosting 是一种试图利用大量弱分类器创建一个强分类器的集成技术。要实现 Boosting 方法，首先你需要利用训练数据构建一个模型，然后创建第二个模型（它企图修正第一个模型的误差）。直到最后模型能够对训练集进行完美地预测或加入的模型数量已达上限，我们才停止加入新的模型。
- AdaBoost 使用浅层决策树。在创建第一棵树之后，使用该树在每个训练实例上的性能来衡量下一棵树应该对每个训练实例赋予多少权重。难以预测的训练数据权重会增大，而易于预测的实例权重会减小。
- 模型是一个接一个依次创建的，每个模型都会更新训练实例权重，影响序列中下一棵树的学习。在构建所有的树之后，我们就可以对新的数据执行预测，并根据每棵树在训练数据上的准确率来对其性能进行加权。
- AdaBoost 是第一个为二分类问题开发的真正成功的 Boosting 算法。它是人们入门理解 Boosting 的最佳起点。当下的 Boosting 方法建立在 AdaBoost 基础之上，最著名的就是随机梯度提升机。

Algorithm Adaboost - Example



courtesy to Alexander Biler <http://sl.ics.uci.edu/Courses/2012F-273a?action=download&upname=10-ensembles.pdf>



2. 机器学习 监督学习步骤

➤ 监督学习是使用已知正确答案的示例来训练网络，每组训练数据有一个明确的标识或结果。如果我们训练一个网络，让其从照片库中（其中包含气球的照片）识别出气球的照片。以下就是我们在这个假设场景中所要采取的步骤。

- **步骤1：数据集的创建和分类**

首先，浏览照片（数据集），确定所有包含气球的照片，并对其进行标注。然后，将所有照片分为训练集和验证集。目标就是在深度网络中找一函数，这个函数输入是任意一张照片，当照片中包含气球时，输出1，否则输出0。

- **步骤2：数据增强 (Data Augmentation)**

一般搜集的数据并不一定包含目标在各种扰动下的信息。数据的好坏对于机器学习模型的预测能力至关重要，因此一般会进行数据增强。对于图像数据来说，数据增强一般包括，**图像旋转，平移，颜色变换，裁剪，仿射变换等**。

- **步骤3：特征工程 (Feature Engineering)**

一般来讲，特征工程包含特征提取和特征选择。常见的手工特征(**Hand-Crafted Feature**)有**尺度不变特征变换(Scale-Invariant Feature Transform, SIFT)**，**方向梯度直方图(Histogram of Oriented Gradient, HOG)**等。由于手工特征是启发式的，其算法设计背后的出发点不同，将这些特征组合在一起的时候有可能会产生冲突，如何将组合特征的效能发挥出来，使原始数据在特征空间中的判别性最大化，就需要用到特征选择的方法。在深度学习方法大获成功之后，人们很大一部分不再关注特征工程本身。因为，最常用到的卷积神经网络(**Convolutional Neural Networks, CNNs**)本身就是一种特征提取和选择的引擎。研究者提出的不同的网络结构、正则化、归一化方法实际上就是深度学习背景下的特征工程。



2. 机器学习 监督学习步骤

- **步骤4：构建预测模型和损失**

将原始数据映射到特征空间之后，下一步就是构建合适的预测模型得到对应输入的输出。而如何保证模型的输出和输入标签的一致性，需要构建模型预测和标签之间的损失函数，常见的**损失函数(Loss Function)**有**交叉熵(Cross Entropy)**、**均方差(MSE)**等。通过优化方法不断迭代，使模型从最初的初始化状态一步步变化为有预测能力的模型的过程，实际上就是学习的过程。

- **步骤5：训练**

选择合适的模型和超参数进行初始化，其中超参数比如支持向量机中核函数、误差项惩罚权重等。当模型初始化参数设定好后，将制作好的特征数据输入到模型，通过合适的优化方法不断缩小输出与标签之间的差距，当迭代到了截止条件，就可以得到训练好的模型。**优化方法最常见的就是梯度下降法及其变种，使用梯度下降法的前提是优化目标函数对于模型是可导的。**

- **步骤6：验证和模型选择**

训练完训练集图片后，需要进行模型测试。利用验证集来验证模型是否可以准确地挑选出含有气球在内的照片。

在此过程中，通常会通过调整和模型相关的各种事物（超参数）来重复步骤2和3，诸如里面有多少个节点，有多少层，使用怎样的激活函数和损失函数，如何在反向传播阶段积极有效地训练权值等等。

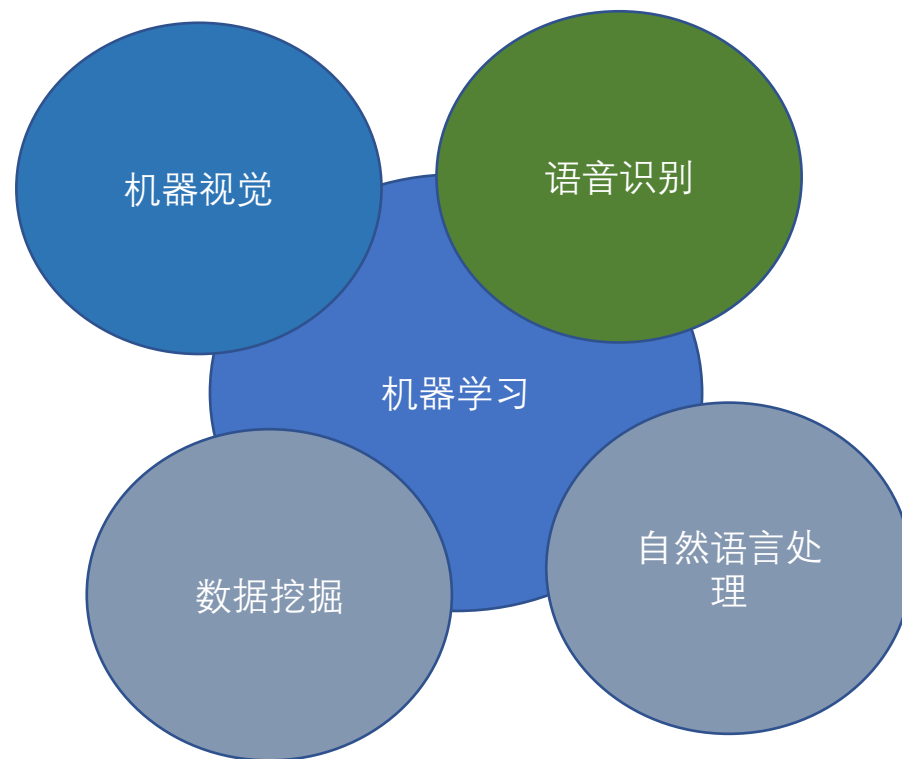
- **步骤7：测试及应用**

当有了一个准确的模型，就可以将该模型部署到你的应用程序中。你可以**将预测功能发布为API（Application Programming Interface, 应用程序编程接口）调用**，并且你可以从软件中调用该API，从而进行推理并给出相应的结果。

2. 机器学习 与机器学习相关的应用领域



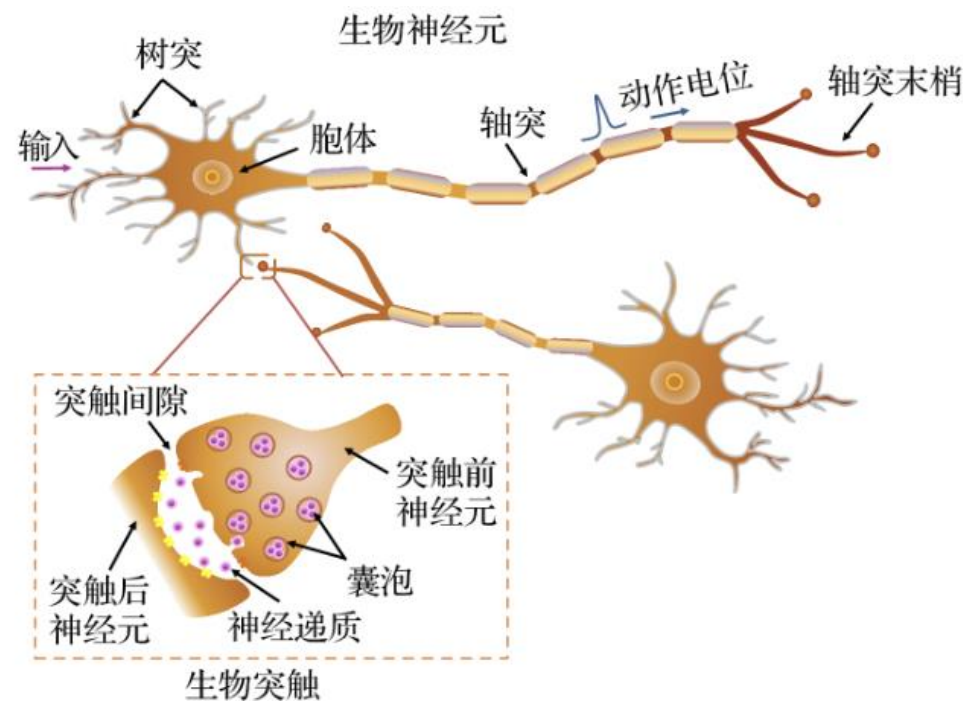
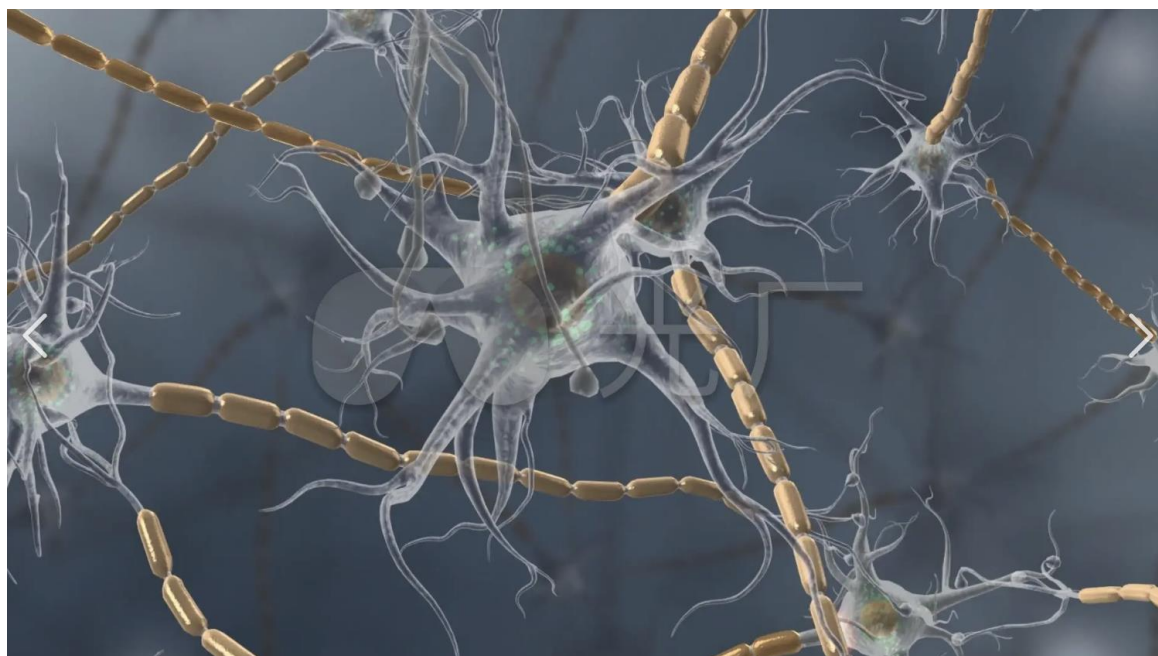
厦门大学
XIAMEN UNIVERSITY



3.1 神经元原理

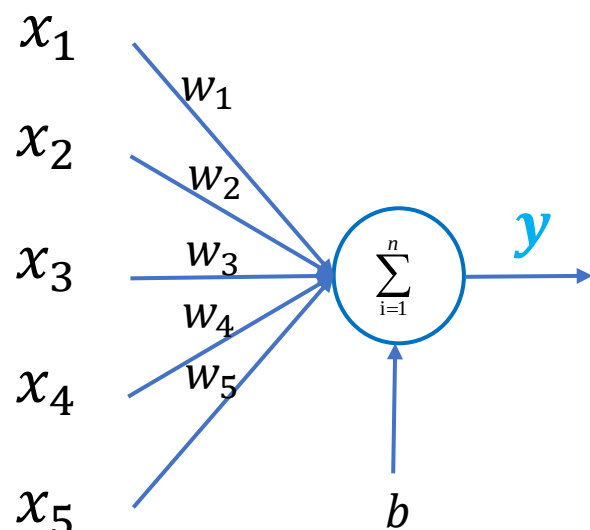


- 人类的大脑由大约800亿个神经元组成，这些神经元由突触与其他神经元相互连接，交换电信号和化学信号。大脑通过神经元之间的协作完成各种功能。神经元之间的连接关系是通过进化、生长发育和后天刺激形成的



- 人工神经网络受动物神经系统的启发，是对其简单的模仿，是感知器模型、logistic回归的进一步发展。

3.2 神经网络结构



➤ 为什么需要激活函数？

线性函数需要加入非线性激活函数获得非线性

➤ sigmoid函数，用于隐层神经元输出，取值范围为(0,1)。可以将一个实数映射到(0,1)的区间，可以用来做二分类，在特征相差比较复杂或是相差不是特别大时效果比较好

$$f(x) = \frac{1}{1 + \exp(-x)}$$

输入值 $(x_1, x_2, x_3, x_4, x_5)$

连接权重 $(w_1, w_2, w_3, w_4, w_5)$

偏置项 b

输出值 $y = f\left(\sum_{i=1}^5 w_i x_i + b\right)$

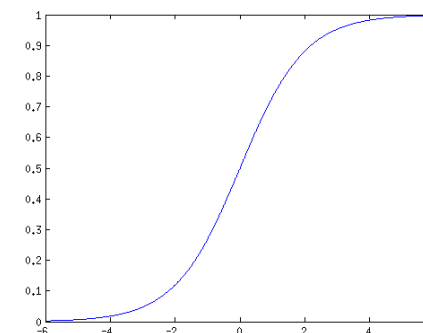
向量和矩阵形式 $f(\mathbf{w}^T \mathbf{x} + b)$

定义域： $(-\infty, +\infty)$

值域： $(0,1)$

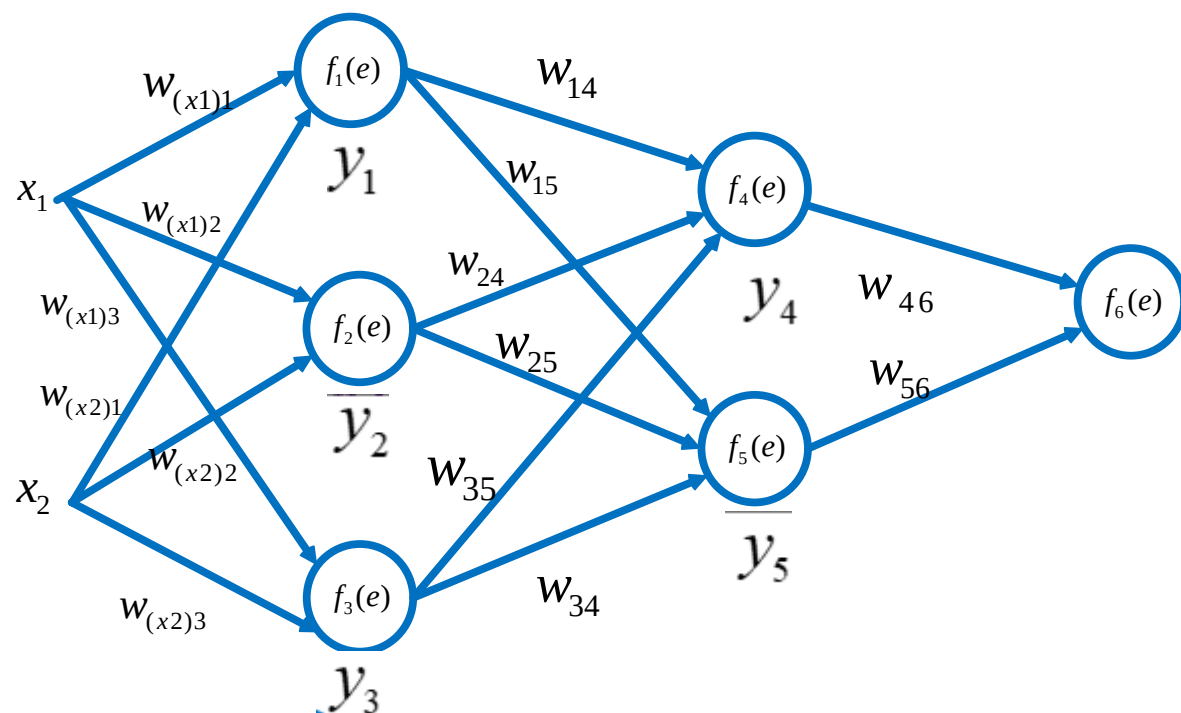
单调性： 递增

导数： $f'(x) = f(x)(1 - f(x))$



3.3 前向传播算法

- 为了方便对前向传播进行说明，我们引入一个2输入，输入层神经元的数量为3，1层隐含层、隐含层神经元的为3，输出层神经元数量为1的全连接神经网络，并对前向传播的过程进行说明



$$y_1 = f_1(w_{(x1)1}x_1 + w_{(x2)1}x_2)$$

$$y_2 = f_2(w_{(x1)2}x_1 + w_{(x2)2}x_2)$$

$$y_3 = f_3(w_{(x1)3}x_1 + w_{(x2)3}x_2)$$

$$y_4 = f_4(w_{14}y_1 + w_{24}y_2 + w_{34}y_3)$$

$$y_5 = f_5(w_{15}y_1 + w_{25}y_2 + w_{35}y_3)$$

$$y = f_6(w_{46}y_4 + w_{56}y_5)$$



3.3 前向传播算法

- 神经网络本质是一多层复合函数，再训练中通过调整权重和偏置项最终 获得不同的映射权重和偏置项的值

- 神经网络实现的是向量到向量的映射 $\mathbb{R}^n \rightarrow \mathbb{R}^m$ $y = h(x)$

- 网络的每一层完成的变换 $u^{(l)} = W^{(l)}x^{(l-1)} + b^{(l)}$

$$x^{(l)} = f(u^{(l)})$$

- 对于前面例子中的网络 $z = f(W^{(2)}f(W^{(1)}x + b^{(1)}) + b^{(2)})$

$$y = f(W^{(1)}x + b^{(1)})$$

$$z = f(W^{(2)}y + b^{(2)})$$

- 完整的前向传播算法

- 设置输入值 $x^{(1)} = x$

- 循环，对于 $l = 2, \dots, m$

$$u^{(l)} = W^{(l)}x^{(l-1)} + b^{(l)}$$

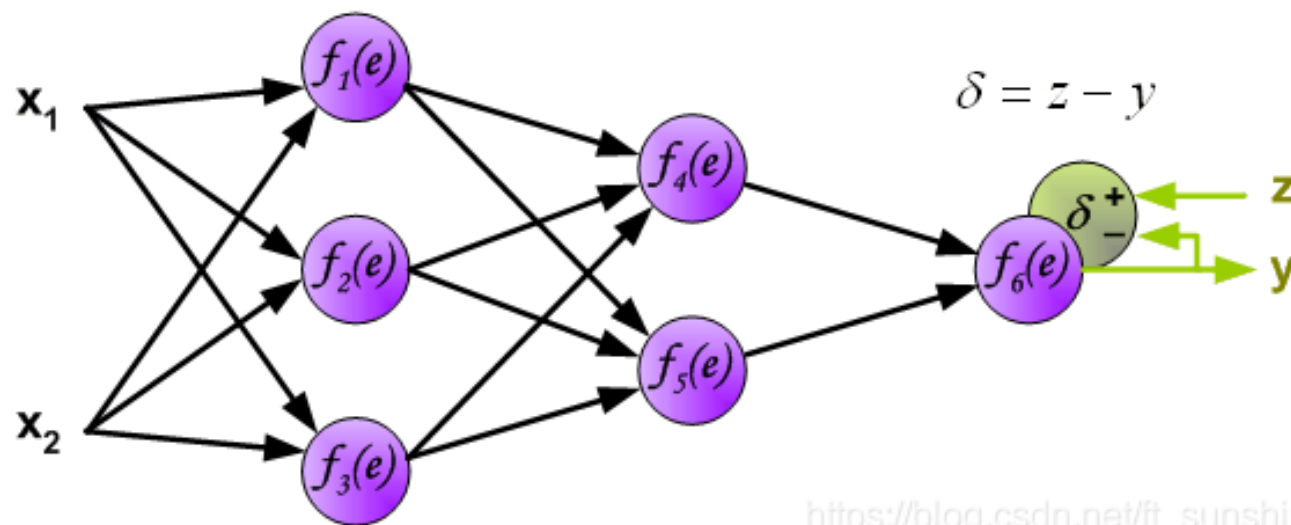
$$x^{(l)} = f(u^{(l)})$$

- 结束循环

- 输出 $x^{(m)}$

3.3 前向传播算法

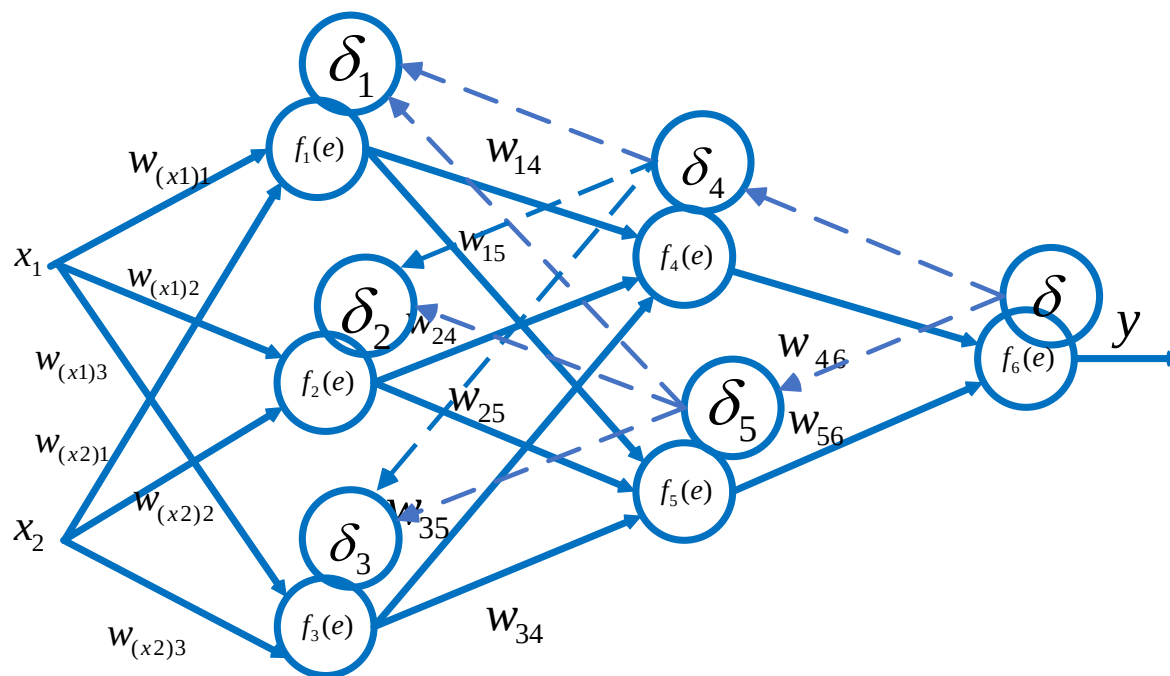
- 神经网络的前向传播完成后输出的 y 就是当次前向传播神经网络计算出来的结果（预测结果）
 - 该预测结果不一定是正确的，要和真实的标签 z 比较
 - 计算预测结果和真实标签存在误差 δ (delta) = $z - y$
- 神经网络参数求导问题源自微积分中多元函数求导的链式法则，其与梯度下降法配合，完成网络训练



3.4 反向传播算法



➤ 误差的反向传播:



$$\delta = z - y$$

$$\delta_4 = w_{46} \delta$$

$$\delta_5 = w_{56} \delta$$

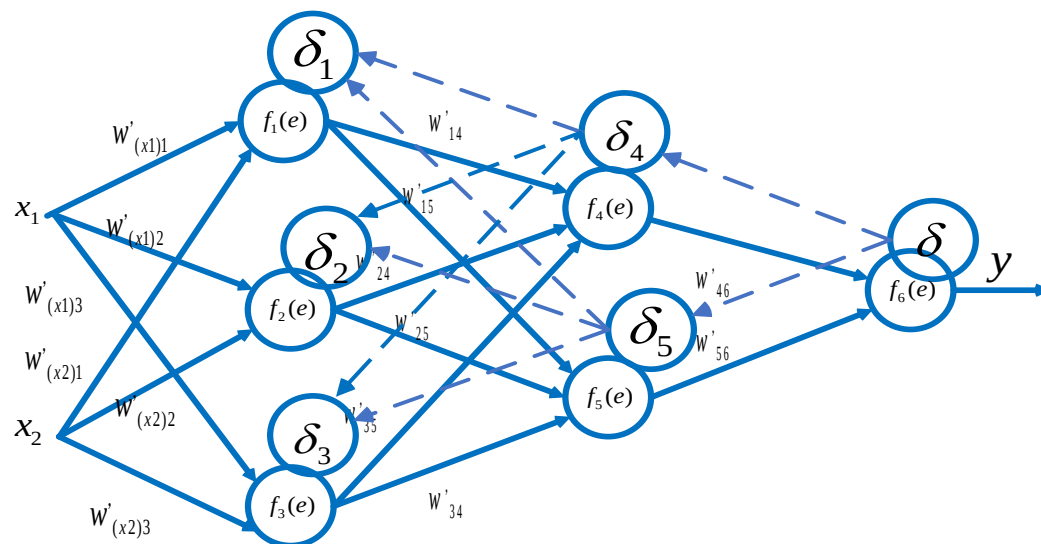
$$\delta_1 = w_{14} \delta_4 + w_{15} \delta_5$$

$$\delta_2 = w_{24} \delta_4 + w_{25} \delta_5$$

$$\delta_3 = w_{34} \delta_4 + w_{35} \delta_5$$

3.4 反向传播算法

- 在实际的计算过程中，需要综合使用求导、矩阵乘法等计算过程实现。
- 在反向传播过程，对于多层网络，其误差曲面存在多个局部极小值，梯度下降可能导致陷入局部极小值。
- 当隐藏层和节点增加时，其权值成倍增长，训练变得极为困难且容易出现过拟合。
- 实践中采用动量法、自适应学习率调整、动量-自适应调整学习率、采用稀疏连接等方法实现训练结果和速度的提升



$$w'_{(x1)1} = w_{(x1)1} + \eta \delta_1 \frac{df_1(e)}{de} x_1$$

$$w'_{(x2)1} = w_{(x2)1} + \eta \delta_1 \frac{df_1(e)}{de} x_2$$

$$w'_{(x1)2} = w_{(x1)2} + \eta \delta_2 \frac{df_2(e)}{de} x_1$$

$$w'_{(x2)2} = w_{(x2)2} + \eta \delta_2 \frac{df_2(e)}{de} x_2$$

$$w'_{(x1)3} = w_{(x1)3} + \eta \delta_3 \frac{df_3(e)}{de} x_1$$

$$w'_{14} = w_{14} + \eta \delta_4 \frac{df_4(e)}{de} y_1$$

$$w'_{24} = w_{24} + \eta \delta_4 \frac{df_4(e)}{de} y_2$$

$$w'_{34} = w_{34} + \eta \delta_4 \frac{df_4(e)}{de} y_3$$

$$w'_{15} = w_{15} + \eta \delta_5 \frac{df_5(e)}{de} y_1$$

$$w'_{25} = w_{25} + \eta \delta_5 \frac{df_5(e)}{de} y_2$$

$$w'_{35} = w_{35} + \eta \delta_5 \frac{df_5(e)}{de} y_3$$

$$w'_{(x2)3} = w_{(x2)3} + \eta \delta_3 \frac{df_3(e)}{de} x_2$$

$$w'_{46} = w_{46} + \eta \delta \frac{df_6(e)}{de} y_4$$

$$w'_{56} = w_{56} + \eta \delta \frac{df_6(e)}{de} y_5$$



3.4 反向传播算法

➤ 欧氏距离 (Euclidean Distance)

二维空间的公式 $\rho = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$, $|X| = \sqrt{x_2^2 + y_2^2}$.

其中, ρ 为点 (x_2, y_2) 与点 (x_1, y_1) 之间的[欧氏距离](#); $|X|$ 为点 (x_2, y_2) 到原点的欧氏距离。

三维空间的公式 $\rho = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$, $|X| = \sqrt{x_2^2 + y_2^2 + z_2^2}$.

n维空间的公式 $d(x, y) := \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$. $|X| = \sqrt{\sum_{i=1}^n x_i^2}$



3.4 反向传播算法

➤ 均方误差(MSE)

数理统计中均方误差是指[参数估计](#)值与参数真值之差平方的期望值，记为MSE。

MSE是衡量“平均误差”的一种较方便的方法，MSE可以评价数据的变化程度。

MSE的值越小，说明预测模型描述实验数据具有更好的精确度。

- SSE(和方差、误差平方和): The sum of squares due to error
- MSE(均方差、方差): Mean squared error
- RMSE(均方根、标准差): Root mean squared error



3.4 反向传播算法

➤ 均方误差(MSE)

- 方差：方差是在概率论和统计方差衡量随机变量或一组数据的离散程度的度量方式，方差越大，离散度越大。求解方式为，各随机变量与平均值差值的平方和的平均数（先求差，再平方，再平均）

平均数：

$$M = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

方差公式：

$$s^2 = \frac{(x_1 - M)^2 + (x_2 - M)^2 + \cdots + (x_n - M)^2}{n}$$

- 标准差：标准差就是方差的算术平方根，它反映组内个体间的离散程度，它的计算过程是与平均值之间进行差值计算。

标准差公式：

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

- 样本方差

$$\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2$$



3.4 反向传播算法

2.均方误差(MSE) 数理统计中均方误差，记为MSE。

SSE(和方差、误差平方和): The sum of squares due to error在统计学里，该参数计算的是拟合数据和原始数据对应点的误差的平方和，计算公式为

$$SSE = \sum_{i=1}^m w_i (y_i - \hat{y}_i)^2$$

其中 y_i 是真实数据 $\hat{y}_i$ 是拟合的数据, $w_i > 0$. SSE越接近于0, 说模型选择和拟合更好, 数据预测也越成功。

MSE(均方差、方差): Mean squared error 也叫回归系统的拟合标准差, 是指[参数估计](#)值与参数真值之差
MSE可以评价数据的变化程度, MSE的值越小, 说明预测模型描述实验数据具有更好的精确度计算公式为

$$MSE = \frac{SSE}{n} = \frac{1}{n} \sum_{i=1}^m w_i (y_i - \hat{y}_i)^2$$

RMSE(均方根、标准差): Root mean squared error也叫回归系统的拟合标准差, 是MSE的平方根, 平方根MSE是衡量“平均误差”的一种较方便的方法, 计算公式为

$$RMSE = \sqrt{MSE} = \sqrt{\frac{SSE}{n}} = \sqrt{\frac{1}{n} \sum_{i=1}^m w_i (y_i - \hat{y}_i)^2}$$



3.5 机器学习模型评价

- 对于二分类问题，可以将样例根据其真实类别与模型预测类别的组合划分为**真正例**（True Positive, TP）、**假正例**（False Positive, FP）、**真反例**（True Negative, TN）、**假反例**（False Negative, FN）四种类别

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

$$TP + FP + TN + FN = Total$$

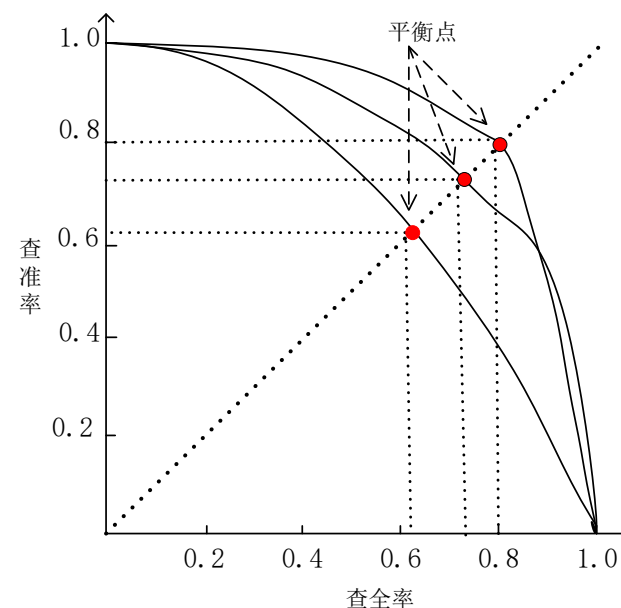
- 查准率**（P，精确率）和**查全率**（R，召回率、**真阳率**）：

$$P = \frac{TP}{TP + FP} \quad \text{预测为正占正例的比例}$$

$$R = \frac{TP}{TP + FN} \quad \text{预测为正的例子中实际为真的比例}$$

- 查准率和查全率是一对矛盾的量，查准率高时，查全率往往偏低，反之亦然。
- 为了有更好的查准率，分类阈值需要设置的较高，提升预测为正例的样本的真正例率，避免假正例，但会导致查不全所有的真正例，查全率偏低；反之亦然。

- 模型的预测结果对样例进行排序，排在前面的最有可能是正例的样例，排在最后面的是最不可能为正例的样例。
- 通过合理设置一个阈值，高于该阈值的预测认为是正例，低于该阈值的预测认为是反例。我们可以得到在该阈值下的该样例的混淆矩阵从而得到一对（P，R）
- 进一步的，可以从高到低的顺序设定阈值，得到不同阈值下的（P，R）对，并根据不同的（P，R）对画出以**查准率为纵轴**、**查全率为横轴**的**P-R曲线**。下图是**P-R曲线**示意图。





3.5 机器学习模型评价

F1值： P-R图显示了模型在总样本上的查全率、查准率。在进行比较时，如果A学习器的P-R曲线把B学习器的P-R曲线包围，则可以知道A学习器的性能优于B学习器。如果B学习器的P-R曲线和C学习器的P-R曲线出现了交叉，则难以比较B、C两个学习器的优劣，只能在具体的P、R条件下比较。如果仍然要对B、C学习器的优劣进行比较，需要有综合权衡P、R的性能量度，F1就是这么一个量度，该值是查准率和查全率的调和平均值，下面是F1的计算式：

$$F1 = \frac{2 * P * R}{P + R} = \frac{2 * TP}{\text{样例数} + TP - TN}$$

不同的运用场景对查准率和查全率的需求是不一样的，推荐系统希望推荐的信息中客户感兴趣的比例尽可能的大，此时更侧重查准率；在信息检索系统中，则是需要检索到的信息尽可能的全面，此时更侧重查全率

F1度量的一般形式为 F_{β} $F_{\beta} = \frac{(1 + \beta^2) \times P \times R}{(\beta^2 \times P) + R}$ 上式中当 $\beta > 1$ 时，侧重查全率的影响；当 $0 < \beta < 1$ 的时候侧重查准率的影响

3.5 机器学习模型评价



当我们进行多次训练和测试时，每次都会有一个对应的混淆矩阵；如果在多个数据集上进行训练和测试时，每个数据集也会有一个混淆矩阵；对应于多分类任务，每两个类别组合也会有对应的混淆矩阵，我们可

(1) 方法1

方法1是直接在各混淆矩阵上计算出对应的查准率、查全率 (P, R) 组合，再计算查全率和查准率的均值及对应的F1值，通过这种方法获得值我们称为宏查准率 (Macro-Precision)、宏查全率 (Macro-Recall) 以及相应的宏F1

(Macro-F1)，具体定义如下：

$$Macro-P = \frac{1}{n} \sum_{i=1}^n P_i$$

$$Macro-R = \frac{1}{n} \sum_{i=1}^n R_i$$

$$Macro-F_1 = \frac{2 \times Macro-P \times Macro-R}{Macro-P + Macro-R}$$

(2) 方法2

方法2是将各混淆矩阵上的对应元素进行平均，再据此计算查全率和查准率的值及对应的F1值，通过这种方法获得值我们称为微查准率 (Micro-Precision)、微查全率 (Micro-Recall) 以及相应的微F1 (Micro-F1)，具体定义如下：

$$Micro-P = \frac{\overline{TP}}{\overline{TP} + \overline{FP}}$$

$$Micro-R = \frac{\overline{TP}}{\overline{TP} + \overline{FN}}$$

$$Micro-F_1 = \frac{2 \times Micro-P \times Micro-R}{Micro-P + Micro-R}$$



3.5 机器学习模型评估

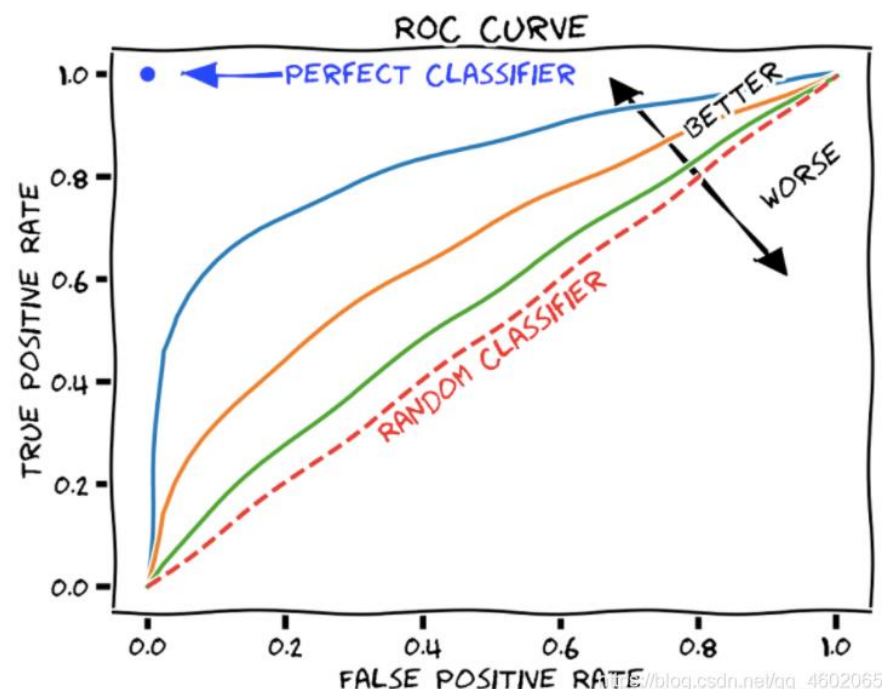
➤ ROC与AUC

- **ROC (Receiver Operating Characteristic) 曲线**，全称是接收者操作特征曲线。ROC曲线产生于第二次世界大战期间，最早用在信号检测领域，侦测战场上的敌军载具（飞机、舰船）。是用来评价二分类模型性能的常用图形工具。
- 它通过显示真阳性率（True Positive Rate, 简称 TPR）与假阳性率（False Positive Rate, 简称 FPR）之间的权衡来帮助我们理解模型的分类能力。
- 其实现分析的方法是一个画在二维平面上的曲线——ROC curve。平面的横坐标是假正例率（False Positive Rate, FPR），纵坐标是真正例率（True Positive Rate, TPR）。
- **AUC (Area Under ROC Curve) 是ROC曲线下的面积**。较大的AUC值表征较好的分类表现（Performance）。
- 通常将单一样本经过模型得到实值或概率预测值后，再将该值与分类阈值（threshold）进行比较，大于该阈值认为是正例，小于该阈值认为是反例。
- 对于同一个样本，经过不同的分类阈值，可以得到不同的正例或者反例的分类结果。这个阈值的设定，会极大的影响模型的泛化能力。

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

$$\text{真阳率 TPR} = \frac{TP}{TP + FN}$$

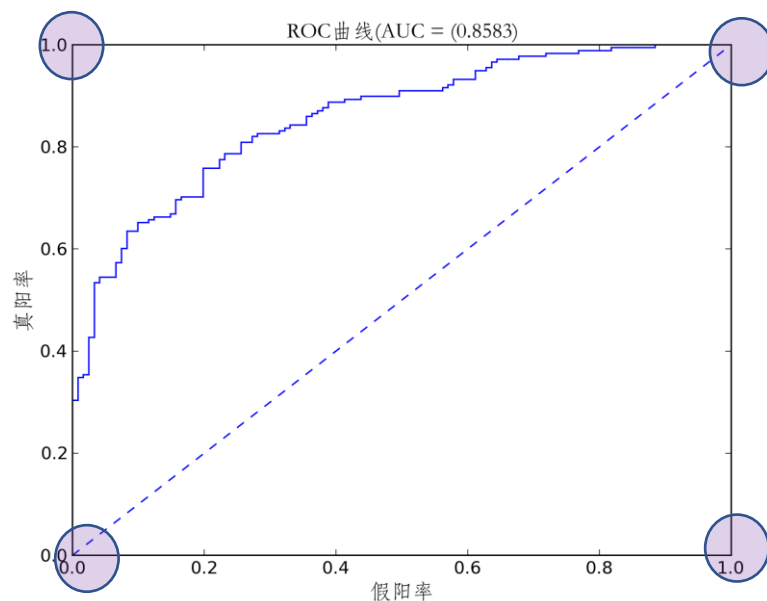
$$\text{假阳率 FPR} = \frac{FP}{FP + TN}$$



3.5 机器学习模型评估



- 将阈值从0-1变化，并根据各变化值计算出该值下的（FPR、TPR）对后，把这些数值对作为坐标点在XOY平面上作图，可以得到ROC曲线。



$$\text{真阳率 TPR} = \frac{TP}{TP + FN}$$

$$\text{假阳率 FPR} = \frac{FP}{FP + TN}$$

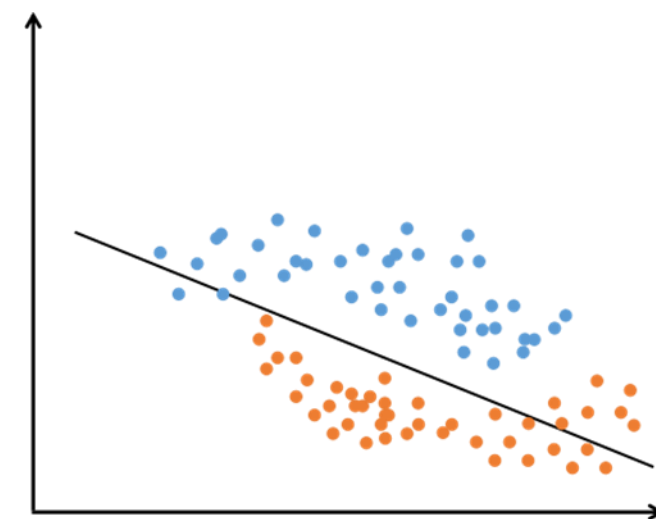
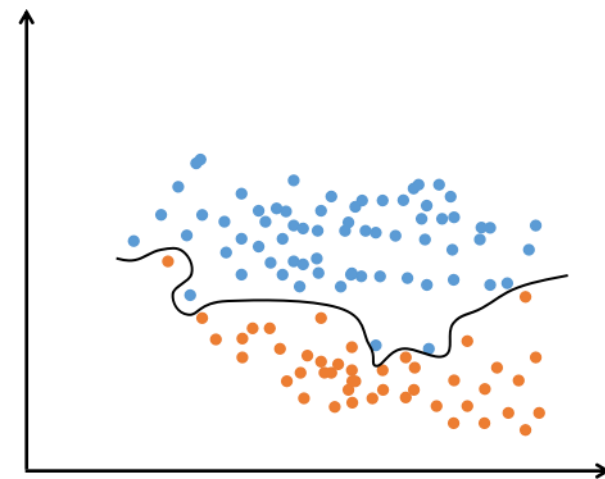
- **ROC曲线图上很有四个点我们需要注意。**
 - **点(0,1)** 此点FPR=0, TPR=1，这表示着FN (False Negative) =0，并且FP (False Positive) =0。意味着这是一个完美的分类器，它将所有的样本都正确分类。
 - **点(1,0)**，此点FPR=1, TPR=0，这是一个糟糕的分类器，因为它成功避开了所有的正确答案。
 - **点(0,0)**，此点FPR=TPR=0，我们可以发现该分类器预测所有的样本都为负样本 (Negative)。
 - **点(1,1)**，此点FPR=TPR=1，分类器实际上预测所有的样本都为正样本。ROC曲线图中有效的曲线是高于y=x的曲线。
- 对于不相交的ROC曲线，越靠近(0,1)的点对应的模型分类性能越好。对于相交的ROC曲线，一般来说AUC值较大的综合性能较好，具体选哪条曲线还需要根据实际的应用需求来确定。
- ROC曲线能够直观地展示分类器在不同阈值下的性能表现。比较不同模型的ROC曲线可以帮助选择最佳模型。也可通过ROC曲线，选择最合适的分类阈值，以满足特定的需求。比如：在医学诊断中，我们可能更重视减少误报率，在搜索引擎中可能更重视提高召回率。
- 通过绘制不同特征集对应的ROC曲线，可以评估不同特征对分类器性能的影响，从而选择最具有区分度的特征集。
- ROC曲线不受类别分布的影响，在处理不平衡样本（正负例数量差异很大）时，ROC曲线能够更好地评估分类器的性能



3.5 机器学习模型评估

➤ 过拟合和欠拟合

- 我们将学习器预测输出与样本真实输出的差异称为**误差**。预测正确的样本数占样本总数比例称为**准确率** (accuracy) , 相反错误样本数占样本总数的比例称为**错误率** (error rate) 。
- 但是准确率并不能有效说明机器学习性能, 实际上达到准确率100%的学习器在大多数情况都不好。实际上我们希望得到的是能够在新样本上表现很好的机器。在新样本上的误差, 我们称为**泛化误差**。
- 训练学习器的时候, 学习器的学习能力太强导致得到的模型对训练集的表现“太好”, 将训练集的特点当成所有样本的普遍规律, 这样会导致泛化性能下降, 这种现象在机器学习被称为 **“过拟合” (Overfitting)** 。
- 过拟合产生的根本原因是**训练数据包含抽样误差**, 在训练时模型将抽样误差也进行了拟合。所谓抽样误差, 是指抽样样本集和整体数据集之间的偏差。
- 引起过拟合的可能原因有: **模型过于复杂**, 拟合样本集中的噪声。需要选更简单的模型或对模型进行裁剪; **样本太少或者缺乏代表性**。此时需要增加样本数, 或者增加样本的多样性; **样本噪声的干扰**, 模型拟合了噪声, 需剔除噪声数据或改用噪声不敏感模型。
- 如果学习器的学习能力太低, 训练集太差, 训练集一般的性质都没有学好, 称为 **“欠拟合” (Underfitting)** 。引起欠拟合的原因: 模型本身过于简单, 例如数据本身是非线性的但使用了线性模型; 特征数太少无法正确的建立映射关系
- 欠拟合问题较为容易解决, 过拟合问题则较难处理。**



3.5 机器学习模型评估



➤ 评估方法

根据模型的泛化能力的不同和计算性能的不同，我们需要比对不同的算法并对同一模型进行参数微调，从而确定最适用于待解决问题的机器学习算法、评估模型的泛化性能和提升预测性能。

- 通常我们可以通过实验测试来对学习器的泛化误差进行评估。
- 我们使用一个不同于训练样本（训练数据集，Training Set）的测试样本（测试集、Testing Set），以模型在测试样本上的测试误差作为泛化误差的近似。
- 为了保证测试样本能符合训练样本的特征分布，测试样例应该从样本的真实分布中独立同分布采样获得且不出现在训练数据集中。
- 在实际工作中，我们通过对原始样本（假设为包含 m 个样例的数据集 $D=\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ）的合理划分，分别得到训练数据集 S 和测试数据集 T 。

➤ 留出法 (Hold-out Method)

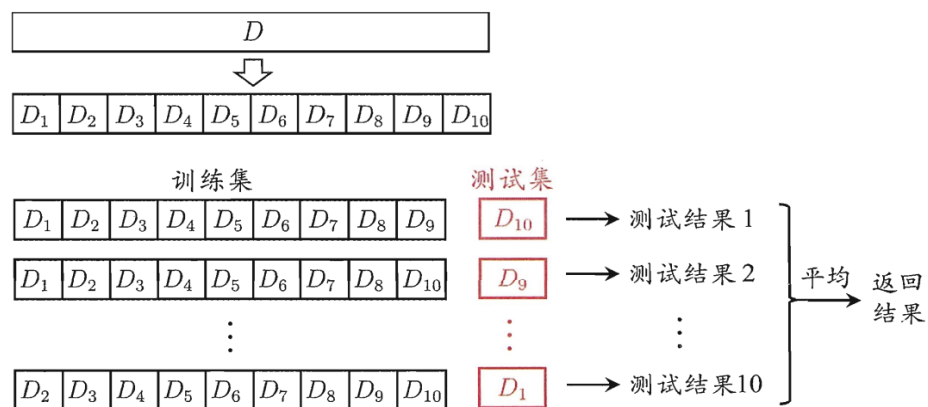
- 留出法直接将数据集 D 划分为两个互斥的部分，其中一部分作为训练集 S ，另一部分用作测试集 T 。
- 通常训练集和测试集的比例为70%：30%。同时，训练集测试集的划分需要注意尽可能保持数据分布的一致性。避免因数据划分过程引入的额外偏差而对最终结果产生影响。
- 在分类任务中，保留类别比例的采样方法称为“分层采样”（stratified sampling）。采用若干次随机划分避免单次使用留出法的不稳定性。

3.5 机器学习模型评估



➤ 交叉验证法 (Cross Validation)

交叉验证法先将数据集 D (样本数量为 m) 划分为 k 个大小相似的互斥子集, 每次采用 $k-1$ 个子集的并集作为训练集, 剩下的那个子集作为测试集。进行 k 次训练和测试, 最终返回 k 个测试结果的均值。又称为“ k 折交叉验证” (K-fold Cross Validation)。为减少因样本划分带来的偏差, 通常重复 p 次不同的划分, 最终结果是 p 次 k 折交叉验证结果的均值。



进一步的, 当 $k=m$ (即数据集中的样本数量 m 与折数 k 相等) 时候为交叉验证的特殊情况, 称为留一法, 即每次只用一个样本作测试集。该方法计算开销较大。

3.5 机器学习模型评价



自助法 (bootstrapping)

自助法以自助采样为基础（有放回采样）。每次随机从D中挑选一个样本，放入D'，然后将样本放回D中，重复m次之后，得到了包含m个样本的数据集。样本在m次采样中始终不被采到的概率是，取极限得到：

$$\lim_{m \rightarrow \infty} (1 - \frac{1}{m})^m \rightarrow \frac{1}{e} \approx 0.368$$

即D中约有36.8%的样本未出现在采样数据集D'中，将D'用在训练集，D\D'用在测试集。

用作测试集。这样使用m个训练样本，约有1/3未出现在训练集中的样本被用作测试集。

自助法在数据集较小、难以有效划分训练/测试集时很有用。同时自助法改变了初始数据集的分布，会引入估计偏差。



3.5 机器学习模型评估

➤ 调参

- 各类学习器中通常都需要进行参数设置，学习获得的模型性能往往随参数设置的不同而不同。在机器学习中，选定了算法后，需要对算法的参数进行设置，这就是我们经常听到的“调参（Parameter Tuning）”。
 - 考虑到每种算法可以调整的参数的数量和各个参数的取值范围，各类参数的取值变化的组合是非常大的，穷尽各种组合的对应的计算开销更是无法实现的。我们通常对每个参数选定一个变化范围和变化步长，在所有的参数的给定的取值范围和变化步长内进行计算选优，在性能较优和可实现性之间进行折中。
- 在调参过程中，参数调整要有针对性，避免过度拟合，参数调整要有耐心，需要多次尝试和比较。通常可以采用如下方法进行调参。
- **手动调参**：使用for循环和交叉验证的策略一步一步逼近最优参数的选取。
 - **网格搜索**：在预先指定的参数范围内生成所有可能的参数组合，然后对每一组参数进行模型训练和评估。
 - **随机搜索**：从超参数空间中随机选择参数组合，参数由固定迭代次数的情况下选择。
 - **贝叶斯搜索**：使用先前对损失的观察结果，以确定下一个（最优）点来抽样

4. 练习



- 1. 理解机器学习、深度学习数学基础
- 2. 理解并掌握全连接神经网络的前向传播和后向传播算法；
- 3. 阅读python_classification.py, 理解基于前向和后向传播的多层感知机训练代码。
- 4. 深刻理解并掌握机器学习、深度学习各项评价指标的含义和计算方法。
- 5. 初步理解验证、评估的意义和方法。